University of Illinois Urbana Champaign

Syntax Semantics Interface
LING 506

Homework 4

Author:
Muhammad Kattom

December 9, 2025



1 TAG Elementary Trees

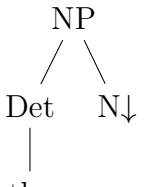
The sentence consists of the lexical items: *John, is, sad, because, the, break, starts, soon.* From how TAGs are made, each item anchors an elementary tree, either an initial tree or an auxiliary tree.

Trees

- α_{john}:



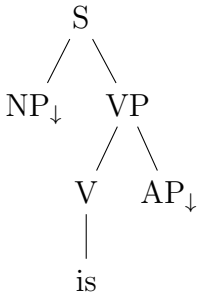
- $\alpha_{\text{the_det}}$:



- $\alpha_{\text{break_n}}$:



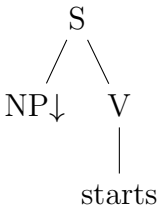
- $\alpha_{\text{is_copula}}$:



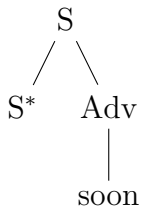
- $\alpha_{\text{sad_adj}}$:



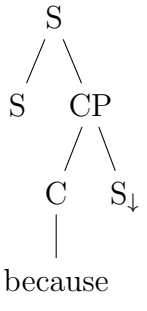
- $\alpha_{\text{start_intrans}}$:



- $\beta_{\text{soon_aux}}$:



- $\alpha_{\text{because_cl}}$:



2 ACG Encoding

We encode TAG derivation structure at the abstract level and TAG derived trees at the object level. A lexicon maps abstract signature to the the object signature.

2.1 Abstract Signature

```
signature abs =
  S, NP, AP, N, Det : type;

  john          : NP;
  the_det       : N -> NP;
  break_n       : N;

  is_copula     : NP -> AP -> S;
  sad_adj       : AP;

  because_cl    : S -> S -> S;

  start_intrans : NP -> S;
  soon_aux      : S -> S;
end
```

This signature represents TAG derivation trees.

2.2 Object Signature

```
signature obj =
  tree : type;
```

```
  john_t, is_t, sad_t, because_t,
  the_t, break_t, starts_t, soon_t : tree;

  np_det_n : tree -> tree -> tree;
  ap_leaf  : tree -> tree;

  s_copula : tree -> tree -> tree;
  s_start_np : tree -> tree;
  s_soon     : tree -> tree;
  s_because  : tree -> tree -> tree;
end
```

2.3 Lexicon Mapping Derivation → Derived Trees

```
lexicon L(abs) : obj =
  S, NP, AP, N, Det := tree;

  john          := john_t;
  the_det       := lambda n. np_det_n the_t n;
  break_n       := break_t;

  sad_adj       := ap_leaf sad_t;

  is_copula     := lambda np. lambda ap. s_copula np ap;

  start_intrans := lambda np. s_start_np np;
  soon_aux      := lambda s. s_soon s;

  because_cl    := lambda s1. lambda s2. s_because s1 s2;
end
```

This lexicon defines how derivation trees are interpreted as derived TAG trees.

3 ACGtk Implementation

The derivation structure for the entire sentence is encoded as:

$$\text{because_cl}\big(\text{is_copula john sad_adj}\big)\big(\text{soon_aux (start_intrans (the_det break_n))}\big).$$

Generation

```
"because_cl (is_copula john sad_adj)
            (soon_aux (start_intrans (the_det break_n))) : S"
| realize lexicons=L;
```

ACGtk Output

```
Term: s_because
        (s_copula john_t (ap_leaf sad_t))
        (s_soon (s_start_np (np_det_n the_t break_t)))
: tree
```

This is exactly the TAG-derived tree.